
First: the mental model 🧠

In Kubernetes, a **Pod** can have **multiple containers**.

Think of a Pod like a **small house**:

- Containers = people living in the house
- They **share the same network** and **storage**

There are two special "helper" types of containers:

1. **Init Containers**
2. **Sidecar Containers**

1️⃣ Init Containers (run before the app)

What is an Init Container?

An **init container** is a container that:

- Runs **before** your main app container
- Must **finish successfully**
- Runs **one by one**
- Then **stops forever**

If an init container fails → the app **never starts**

Why do we need init containers?

Because sometimes your app **is not ready to start yet**.

Common reasons:

- Database isn't ready
- Config file not downloaded
- Permissions not set
- Waiting for another service

Simple real-life analogy 🚪

You can't start cooking:

- ❌ Before buying groceries
- ❌ Before cleaning the kitchen

Init containers = **preparation steps**

Basic flow (lad steps 🪜)

1. Pod is created
2. Init container #1 runs
3. Init container #1 finishes ✅
4. Init container #2 runs (if any)
5. All init containers finish
6. Main app container starts 🚀

Very basic example

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  initContainers:
```

```
- name: wait-for-db
  image: busybox
  command: ['sh', '-c', 'until nslookup mydb; do sleep 2; done']

containers:
- name: my-app
  image: nginx
```

👉 **Meaning**:

- wait-for-db runs first
- It waits until mydb exists
- Only then does nginx start

Key points to remember 🧠

✅ Runs **before** app
✅ Runs **once**
✅ Must **finish**
❌ Cannot run alongside the app

2️⃣ Sidecar Containers (run with the app)

What is a Sidecar Container?

A **sidecar container**:

- Runs **at the same time** as your main app
- Stays alive **as long as the app runs**
- Helps the app while it's running

Why do we need sidecars?

Your app might need:

- Logging
- Monitoring
- Proxying traffic
- Config updates

But you don't want this logic inside your app code.

Simple real-life analogy 🚗

Your app = driver
Sidecar = GPS / dashcam / music system

They:

- Start together
- Work together
- Stop together

Basic flow (lad steps 🪜)

1. Pod starts
2. Main container starts
3. Sidecar container starts
4. Both run together
5. If Pod stops → both stop

Very basic example

apiVersion: v1

```
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: my-app
    image: nginx

  - name: log-collector
    image: busybox
    command: ['sh', '-c', 'tail -f /var/log/nginx/access.log']
```

👉 **Meaning**:

- nginx serves traffic
- log-collector reads logs at the same time

Key points to remember 🧠

✅ Runs **with** app
✅ Long-running
✅ Shares network & storage
❌ Not a one-time job

🔥 **Init vs Sidecar (super simple table)**

Feature	Init Container	Sidecar Container
When it runs	Before app	With app
Runs once?	Yes	No

Must finish?	Yes	No
Typical use	Setup / wait	Logging / proxy
App depends on it	Yes	Yes (runtime)

When to use what? 🤔

Use Init Container when:

- Something must happen **before** app starts
- Example: wait for DB, download config

Use Sidecar Container when:

- Something must run **alongside** app
- Example: logging, service mesh proxy
